POLITÉCNICA

Estrategias de diseño basadas en modelos y verificación formal

José M. Moya

Departamento de Ingeniería Electrónica

Objetivos Tema 1

- 1.1 Contexto actual
- 1.2 Características y arquitecturas de redes de sensores
  - WSN
  - M2M communications
  - CPS
- 1.3 Paradigma IoT
- 1.4 Gestión de recursos y servicios en nodos intermedios
  - Edge fog computing
- **1.5 Estrategias de diseño: basadas en modelos y verificación formal**

Model Based Design (MBD)

Implementación de Sistemas Empotrados

a) Normalmente: Programar directamente sin pensar

b) SEMP + SEMA: Estrategia de **Ingeniería de Software**

Model Based Design (MBD)

POLITÉCNICA

Implementación de Sistemas Empotrados

a) Normalmente: Programar directamente sin pensar

b) RESE: Estrategia de **Ingeniería de Software**

Diseño basado en modelos / Model Based Design (MBD)

Safety → El sistema siempre hace lo que tiene que hacer

Fiabilidad → El sistema funciona incluso en el caso peor

Implementación automática → Modelado formal

Model Based Design (MBD)

1	2	3	4	5	6
Especificación	Modelado	Verificación	Implementación	Verificación	Análisis
¿**PARA QUÉ**, cuál es el problema?	¿**QUÉ** tiene que hacer el sistema?	¿El modelo **CUMPLE** la especificación?	¿**CÓMO** hace lo que tiene que hacer? (modelo refinado)	¿La implementación **CUMPLE** la especificación?	¿Se cumplen los **PLAZOS** en el caso peor?
LTL	FSM / xFSM	Spin (model checking)	Ejecutivo cíclico / Threads FreeRTOS	Spin (model checking)	Análisis del tiempo de respuesta

Model Based Design (MBD)

- Práctica 1

1	2	3	4	5	6
Especificación	Modelado	Verificación	Implementación	Verificación	Análisis
¿**PARA QUÉ**, cuál es el problema?	¿**QUÉ** tiene que hacer el sistema?	¿El modelo **CUMPLE** la especificación?	¿**CÓMO** hace lo que tiene que hacer? (modelo refinado)	¿La implementación **CUMPLE** la especificación?	¿Se cumplen los **PLAZOS** en el caso peor?
LTL	FSM / xFSM	Spin (model checking)	Ejecutivo cíclico / Threads FreeRTOS	Spin (model checking)	Análisis del tiempo de respuesta

Model Based Design (MBD)

- Práctica 2 → Tema 3

1	2	3	4	5	6
Especificación	Modelado	Verificación	Implementación	Verificación	Análisis
¿**PARA QUÉ**, cuál es el problema?	¿**QUÉ** tiene que hacer el sistema?	¿El modelo **CUMPLE** la especificación?	¿**CÓMO** hace lo que tiene que hacer? (modelo refinado)	¿La implementación **CUMPLE** la especificación?	¿Se cumplen los **PLAZOS** en el caso peor?
LTL	FSM / xFSM	Spin (model checking)	Ejecutivo cíclico / Threads FreeRTOS	Spin (model checking)	Análisis del tiempo de respuesta

Objetivos → Práctica 1

1.5 Estrategias de diseño: basadas en modelos y verificación formal

- Especificación → LTL
- Modelos → FSM Mealy
- Verificación formal → Spin

Objetivos → Práctica 1

- **LTL** (Linear Temporal Logic)
  - Describe las relaciones causales del sistema
- **Promela**
  - Lenguaje de especificación de programas concurrentes
  - Modelo promela del sistema
- **SPIN**
  - Herramienta SW de verificación formal open-source

Especificaciones → LTL
Modelo sistema → FSM
promela → spin

Especificación con LTL

Model Based Design (MBD)

1	2	3	4	5	6
Especificación	Modelado	Verificación	Implementación	Verificación	Análisis
¿PARA QUÉ, cuál es el problema?	¿**QUÉ** tiene que hacer el sistema?	¿El modelo **CUMPLE** la especificación?	¿**CÓMO** hace lo que tiene que hacer? (modelo refinado)	¿La implementación **CUMPLE** la especificación?	¿Se cumplen los **PLAZOS** en el caso peor?
LTL	FSM / xFSM	Spin (model checking)	Ejecutivo cíclico / Threads FreeRTOS	Spin (model checking)	Análisis del tiempo de respuesta

Especificación LTL

- Especificación

- Verificación

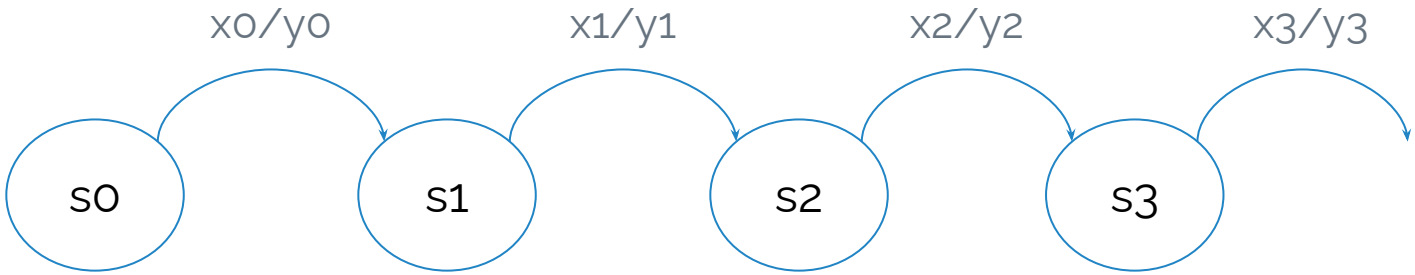
Especificación LTL

- Especificación
  - Declaración del objetivo del diseño **sin ambigüedades**
  - **Propiedades** deseadas del sistema
- Verificación
  - ¿El sistema diseñado **alcanza su objetivo** en el entorno?
  - ¿**Cumple todas las propiedades** deseadas?

Propiedades = Afirmaciones sobre las trazas de ejecución

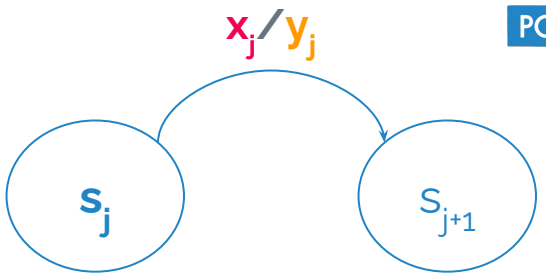
Especificación LTL

- LTL: Linear Temporal Logic
  - **Traza de ejecución** de una FSM
    - q0, q1, q2, q3, … donde
      - qj = (xj,sj,yj) es una reacción
      - xj es la condición de guarda en el paso j
      - sj es el estado en el paso j
      - yj es la actuación en el paso j



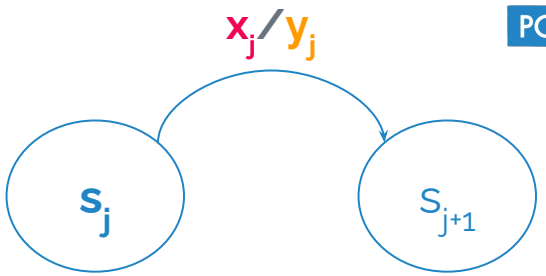
Especificación LTL

- LTL: Linear Temporal Logic
  - **Proposiciones atómicas**
    - Enunciados sobre una reacción $q_j = (x_j, s_j, y_j)$



Atomic proposition	Promela	
x1	x1	True if input x1 is present
x1 = present	(x1==1)	True if input x1 is present
y1 = absent	(y1==0)	True if y1 is absent
s1	(*my_state*==s1)	True if the FSM is in state s1

Especificación LTL

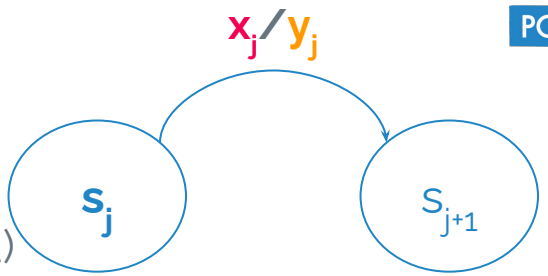


- LTL: Linear Temporal Logic
  - **Proposiciones atómicas**
    - Enunciados sobre una reacción $q_j = (x_j, s_j, y_j)$

Atomic proposition	Promela	
x1	x	rue if input x1 is present
x1 = present		ue if input x1 is present
y1 = absent	y1	rue if y1 is absent
s1	(my_sta ==s1	True if the FSM is in state s1

La expresión es true o false en una reacción q_j

Especificación LTL



- LTL: Linear Temporal Logic
  - **Proposiciones** (proposition logic formula)
    - Combina proposiciones atómicas con operadores lógicos

Proposition	Promela	
x1 ∧ y1	x1 && y1	True if x1 and y1 are **both present**
x1 ∨ y1	x1 || y1	True if either **x1 or y1 is present**
¬y1	!y1	True if y1 i**s absent**
s1 ⇒ y1	s1 -> y1	True if whenever the FSM is in state s1, the output y1 will be made present by the reaction (q1)

Especificación LTL

- LTL: Linear Temporal Logic
  - **Proposici**
    - Comb

x_j / y_j

Si p1 y p2 son proposiciones:

p1 ⇒ p2 es true si y sólo si **¬p2 ⇒ ¬p1**

p1 ⇒ p2 es true si p1 es false
(¬p2 ⇒ ¬p1 es true independientemente de p2 si ¬p1 es true)

p1 ⇒ p2 es lo mismo que **¬p1 ∨ p2**

Proposition	Pron	
x1 ∧ y1	x1 &	
x1 ∨ y1		
¬y1	!y1	
s1 ⇒ y1	s1 -> y1	True if whenever the FSM is in state s1, the output y1 will be made present by the reaction (q1)

Especificación LTL

- LTL: Linear Temporal Logic
  - **Fórmulas LTL** → para especificar **propiedades del sistema**
    - Afirmaciones sobre las trazas de ejecución completas
    - Notación:
      - Proposiciones: p, p1, p2, …
      - Fórmulas LTL: φ, φ1, φ2, …

Especificación LTL

- LTL: Linear Temporal Logic
  - **Fórmulas LTL** → para especificar **propiedades del sistema**
    - Afirmaciones sobre las trazas de ejecución completas
    - Notación:
      - Proposiciones: p, p1, p2, …
      - Fórmulas LTL: φ, φ1, φ2, …

Proposiciones: enunciados sobre una reacción q_j
Fórmulas LTL: enunciados sobre la traza de ejecución completa $q_0, q_1, q_2, \ldots$

Especificación LTL

- LTL: Linear Temporal Logic
  - **Fórmulas LTL** → para especificar **propiedades del sistema**
    - Afirmaciones sobre las trazas de ejecución completas
    - Notación:
      - Proposiciones: p, p1, p2, …
      - Fórmulas LTL: φ, φ1, φ2, …
        - **Pueden estar compuestas de proposiciones y de operadores temporales**

Especificación LTL

- LTL: Linear Temporal Logic
  - **Fórmulas LTL** → para especificar **propiedades del sistema**
    - Afirmaciones sobre las trazas de ejecución completas

Operadores temporales	Textual	Symbolic	Promela
Globalmente	G	$\Box$	[]
Eventualmente en el futuro	F	$\Diamond$	<>
Until	U	$\mathcal{U}$	U
Weak until	W	$\mathcal{W}$	W

Especificación LTL

Siempre

- LTL: Linear Temporal Logic
  - **Fórmulas LTL** → para especificar **propiedades del sistema**
    - Afirmaciones sobre las trazas de ejecución completas

Operadores temporales	Textual	Symbolic	Promela
Globalmente	G	$\Box$	[]
Eventualmente en el futuro	F	$\Diamond$	<>
Until	U	$\mathcal{U}$	U
Weak until	W	$\mathcal{W}$	W

Especificación LTL

- LTL: Linear Temporal Logic
  - **Fórmulas LTL** → para especificar **propiedades del sistema**
    - Afirmaciones sobre las trazas de ejecución completas

En algún momento

Operadores temporales	Textual	Symbolic	Promela
Globalmente	G	$\Box$	[]
Eventualmente en el futuro	F	$\Diamond$	<>
Until	U	$\mathcal{U}$	U
Weak until	W	$\mathcal{W}$	W

Especificación LTL

POLITÉCNICA

- LTL: Linear Temporal Logic
  - **Fórmulas LTL** → para especificar **propiedades del sistema**
    - Afirmaciones sobre las trazas de ejecución completas

Hasta que se cumple otra propiedad

Operadores temporales	Textual	Symbolic	Promela
Globalmente	G	$\Box$	[]
Eventualmente en el futuro	F	$\Diamond$	<>
Until	U	$\mathcal{U}$	U
Weak until	W	$\mathcal{W}$	W

Especificación LTL

Hasta que se cumple otra propiedad (si se cumple)

- LTL: Linear Temporal Logic
  - **Fórmulas LTL** → para especificar **propiedades del sistema**
    - Afirmaciones sobre las trazas de ejecución completas

Operadores temporales	Textual	Symbolic	Promela
Globalmente	G	$\Box$	[]
Eventualmente en el futuro	F	$\Diamond$	<>
Until	U	$\mathcal{U}$	U
Weak until	W	$\mathcal{W}$	W

Especificación LTL

- LTL: Linear Temporal Logic

Op. temporales	q0 q1 q2 q3 q4 …	Fórmula LTL es true si …
G φ	φ φ φ φ φ	φ se cumple para todos los sufijos
F φ	φ	φ se cumple para algún sufijo
φ1 U φ2	φ1 φ1 φ1 φ2	φ2 se cumple para algún sufijo, y φ1 se cumple hasta que φ2 es true
φ1 W φ2	φ1 φ1 φ1 φ2 φ1 φ1 φ1 φ1 φ1	φ1 se cumple hasta que φ2 es true, pero no es necesario que se de φ2

Especificación LTL

- LTL: Linear Temporal Logic
  - **Fórmulas LTL** → para especificar **propiedades del sistema**
    - Afirmaciones sobre las trazas de ejecución completas

Dada una FSM M y una fórmula LTL φ

φ se cumple para M si φ se cumple
para todas las posibles trazas de ejecución de M

(Consideración de todas las posibles entradas)

Especificación LTL

- LTL: Linear Temporal L
  - **Fórmulas LTL** → para e
    - Afirmaciones sobre

Para que una FSM cumpla una propiedad no es suficiente con que una traza de ejecución cumpla la propiedad

Todas y cada una de las posibles trazas de ejecución tienen que cumplir la propiedad

Dada una FSM M y u ... ula LTL φ

φ <u>se cumple</u> para M si φ se cumple
para todas las posibles trazas de ejecución de M

(Consideración de todas las posibles entradas)

Especificación LTL

POLITÉCNICA

- LTL: Linear Temporal Logic
  - **Fórmulas LTL** → para especificar **propiedades del sistema**
    - Afirmaciones sobre las trazas de ejecución completas

Dada una FSM M y una fórmula LTL φ

Para demostrar que φ no se cumple para M
sólo es necesario encontrar un contraejemplo

(Una traza para la que el enunciado es falso)

Especificación LTL

- LTL: Linear Temporal L
  - **Fórmulas LTL** → para e
    - Afirmaciones sobre

Si la propiedad no se cumple para alguna traza de ejecución de la FSM (existe un **contraejemplo**), entonces

la FSM no cumple la propiedad

Dada una FSM M y u ula LTL φ

Para demostrar que φ no se cumple para M
sólo es necesario encontrar un contraejemplo

(Una traza para la que el enunciado es falso)

Especificación LTL

- LTL: Linear Temporal Logic
  - Fórmulas LTL de interés
    - **Request-response**
    - **Strong fairness**
    - **Safety**
  - Catálogo de patrones de fórmulas LTL
    - https://matthewbdwyer.github.io/psp/patterns/ltl.html

Especificación LTL

- Fórmula LTL Request-response: **G(p ⇒ Fq)**
  - Si se cumple una condición entonces se genera una acción
    - Siempre que (condición → en algún momento acción)
    - Globalmente (condición → eventualmente acción)
    - G (condición ⇒ F acción)
    - **[] (condición -> <> acción)**

Demostrar que el sistema responde

Especificación LTL

- Fórmula LTL Request-response: **G(p ⇒ Fq)**
  - Si se cumplen **c1 y c2** entonces se genera una acción
    - Siempre que (c1 y c2 → en algún momento acción)
    - Globalmente (c1 y c2 → eventualmente acción)
    - G (c1 ∧ c2 ⇒ F acción)
    - **[] ((c1 && c2) -> <> acción)**

Especificación LTL

- Fórmula LTL Request-response: **G(p ⇒ Fq)**
  - Si se cumplen **c1 o c2** entonces se genera una acción
    - Siempre que (c1 o c2 → en algún momento acción)
    - Globalmente (c1 o c2 → eventualmente acción)
    - G (c1 ∨ c2 ⇒ F acción)
    - **[] ((c1 | | c2) -> <> acción)**

Especificación LTL

Siempre se cumple que en algún momento p es true

- Fórmula LTL Strong fairness: **G Fp ⇒ G Fq**
  - Todo proceso habilitado infinitamente a menudo se ejecutará infinitamente a menudo
    - Si infinitamente a menudo condición → infinitamente a menudo acción
    - G F condición ⇒ G F acción
    - **[]<> condición → []<> acción**

Demostrar que el sistema está vivo (alive) y no se atasca

Especificación LTL

- Fórmula LTL Safety: **G (¬p)**
  - Nunca se puede dar P
    - No se pueden dar las condiciones c1 y c2 simultáneamente
    - G ¬(c1 ∧ c2)
    - **[] (! (c1 && c2))**

Asegurar condiciones de seguridad

Especificación LTL

Práctica A

- **Especificación**
  - Un pasillo con una bombilla controlada por un pulsador
  - Si la luz está apagada, activar el pulsador enciende la luz
  - Si la luz está encendida, activar el pulsador apaga la luz

Especificación de las propiedades del sistema

Modelado con FSM de Mealy

Model Based Design (MBD)

1	2	3	4	5	6
Especificación	Modelado	Verificación	Implementación	Verificación	Análisis
¿**PARA QUÉ**, cuál es el problema?	¿**QUÉ** tiene que hacer el sistema?	¿El modelo **CUMPLE** la especificación?	¿**CÓMO** hace lo que tiene que hacer? (modelo refinado)	¿La implementación **CUMPLE** la especificación?	¿Se cumplen los **PLAZOS** en el caso peor?
LTL	FSM / xFSM	Spin (model checking)	Ejecutivo cíclico / Threads FreeRTOS	Spin (model checking)	Análisis del tiempo de respuesta

Model Based Design (MBD)

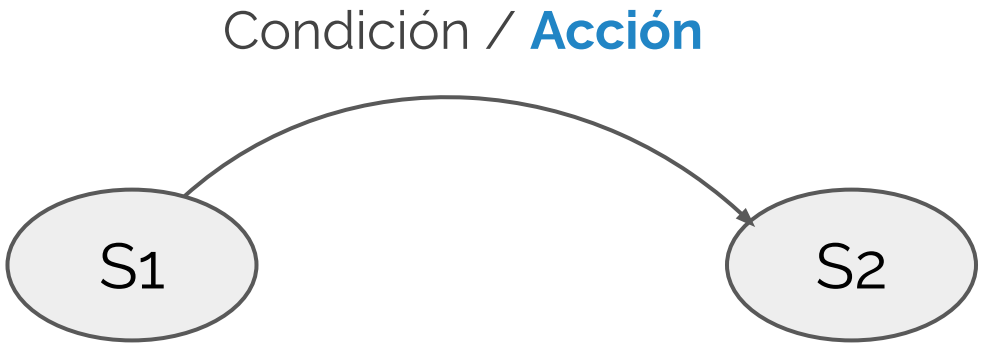
- ¿Qué es un modelo?
  - Representación abstracta, física, matemática
  - … de sistemas, procesos
  - … que permite describirlos, analizarlos, predecirlos

Nuestra representación → FSM Mealy

(FSM = Finite State Machine)

MBD: FSM Mealy

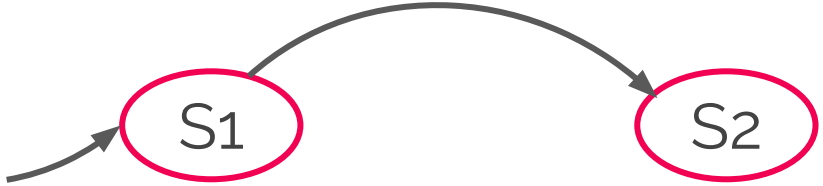
- Máquinas de estados de Mealy
  - **Acciones asociadas a la transición**



MBD: Formalismo FSM

Estado: Forma única y diferente de reaccionar
(resumen del comportamiento del sistema)

Condición / Acción



- Inputs
- Outputs
- Estados = {S0, S1, S2, …, SM}
- Estado inicial = S1
- Update function = Estados x Inputs → Estados x Outputs

Definidos gráficamente FSM

MBD: Formalismo FSM

Transición: Reacción ante un evento
(cualquier cambio en señal)

Condición / Acción

Condición de guarda → Inputs

Acción → Valor de outputs

- Inputs
- Outputs
- Estados = {S0, S1, S2, ..., SM}
- Estado inicial = S1
- Update function = Estados x Inputs → Estados x Outputs

Definidos gráficamente FSM

MBD: Formalismo FSM

Transición: Reacción ante un evento
(cualquier cambio en señal)

Condición / Acción

- Inputs
- Outputs
- Estados = {S0, S1, S2, ..., SM}
- Estado inicial = S1
- Update function = Estados x Inputs → Estados x Outputs

Definidos gráficamente FSM

Extended FSM

Aumentar el modelo con variables

Se pueden leer y escribir en las transiciones

MBD: Formalismo FSM

Transición: Reacción ante un evento
(cualquier cambio en señal)

Condición / Acción

- Inputs
- Outputs
- Estados = {S0, S1, S2, …, SM}
- Estado inicial = S1
- Update function = Estados x Inputs → Estados x Outputs

No realizar cambios en la evaluación de la condición

Ni en variables ni en salidas

Se puede evaluar y no cumplir la condición

gráficamente FSM

MBD: Formalismo FSM

Transición: Reacción ante un evento
(cualquier cambio en señal)

Condición / Acción

Realizar cambios sólo en acciones

Asignación de nuevos valores en salidas o variables

- Inputs
- Outputs
- Estados = {S0, S1, S2, ..., SM}
- Estado inicial = S1
- Update function = Estados x Inputs → Estados x Outputs

Definidos gráficamente FSM

MBD: Formalismo FSM

Transición: Reacción ante un evento
(cualquier cambio en señal)

FSM síncrona
Función update se dispara periódicamente

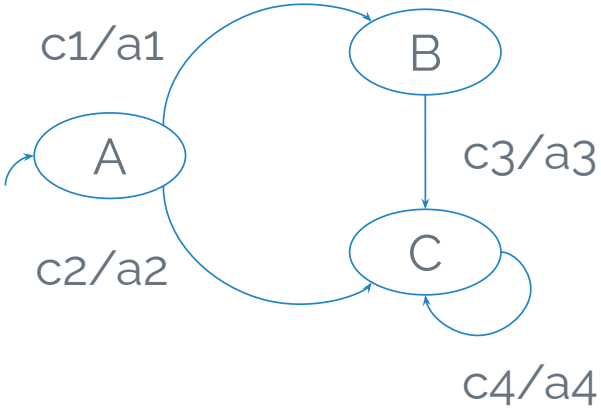
- Inputs
- Outputs
- Estados = {S0, S1, S2, …, SM}
- Estado inicial = S1
- Update function = Estados x Inputs → Estados x Outputs

Definidos gráficamente FSM

Modelado: FSM síncrona

Update → fsm_fire(...)→ avanza la FSM síncrona

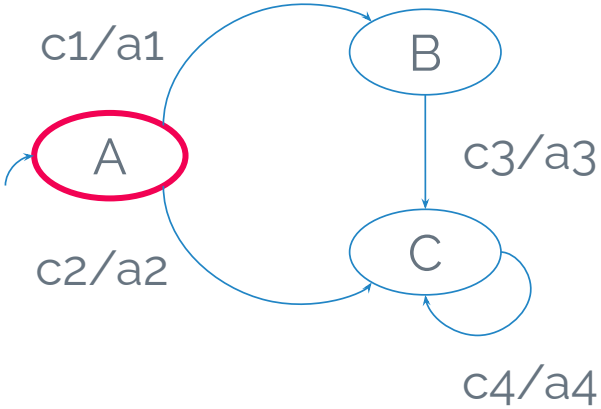
1. Comprueba en qué estado está
2. Qué posibles transiciones existen
3. Evalúa las condiciones
4. De la transición que se cumpla:
   acción + actualización del estado
5. Si no se cumple ninguna no avanza



Modelado: FSM síncrona

Update → fsm_fire(...)→ avanza la FSM síncrona

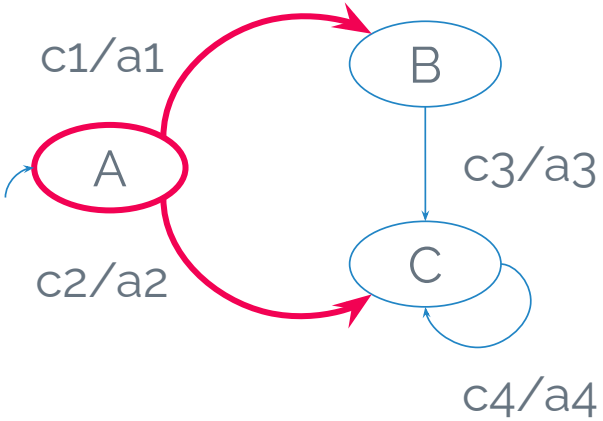
1. Comprueba en qué estado está
2. Qué posibles transiciones existen
3. Evalúa las condiciones
4. De la transición que se cumpla:
   acción + actualización del estado
5. Si no se cumple ninguna no avanza



Modelado: FSM síncrona

Update → fsm_fire(...)→ avanza la FSM síncrona

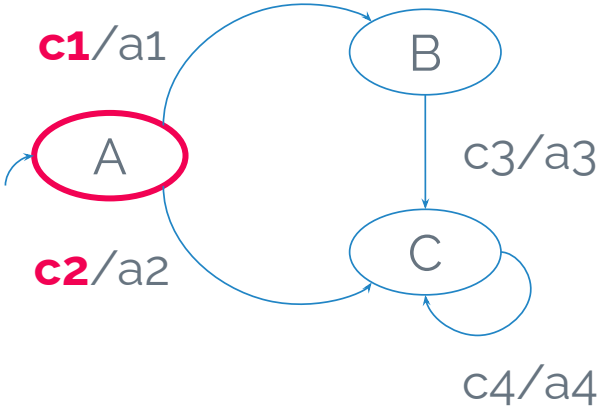
1. Comprueba en qué estado está
2. Qué posibles transiciones existen
3. Evalúa las condiciones
4. De la transición que se cumpla:
   acción + actualización del estado
5. Si no se cumple ninguna no avanza



Modelado: FSM síncrona

Update → fsm_fire(...)→ avanza la FSM síncrona

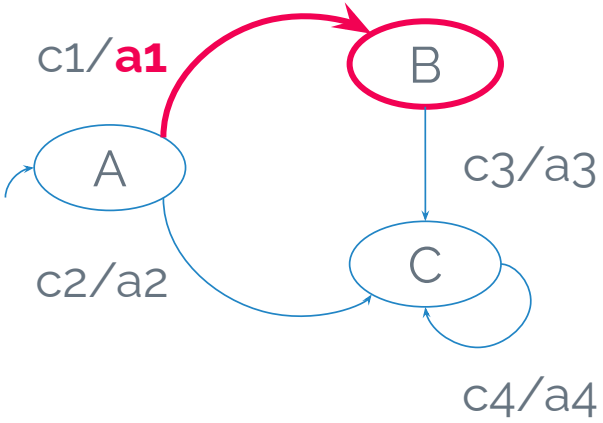
1. Comprueba en qué estado está
2. Qué posibles transiciones existen
3. Evalúa las condiciones
4. De la transición que se cumpla:
   acción + actualización del estado
5. Si no se cumple ninguna no avanza



Modelado: FSM síncrona

Update → fsm_fire(...)→ avanza la FSM síncrona

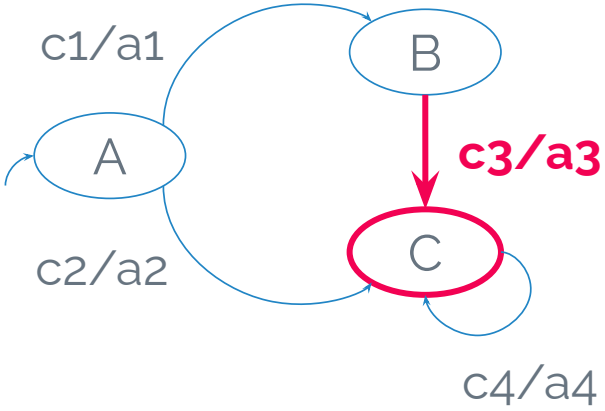
1. Comprueba en qué estado está
2. Qué posibles transiciones existen
3. Evalúa las condiciones
4. De la transición que se cumpla:
   acción + actualización del estado
5. Si no se cumple ninguna no avanza



Modelado: FSM síncrona

Update → fsm_fire(...)→ avanza la FSM síncrona

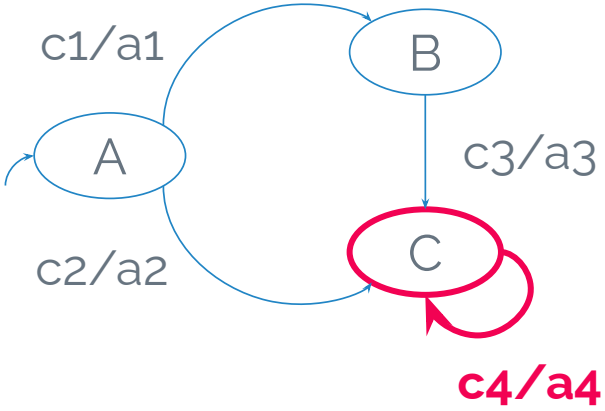
1. Comprueba en qué estado está
2. Qué posibles transiciones existen
3. Evalúa las condiciones
4. De la transición que se cumpla:
   acción + actualización del estado
5. Si no se cumple ninguna no avanza



Modelado: FSM síncrona

Update → fsm_fire(...)→ avanza la FSM síncrona

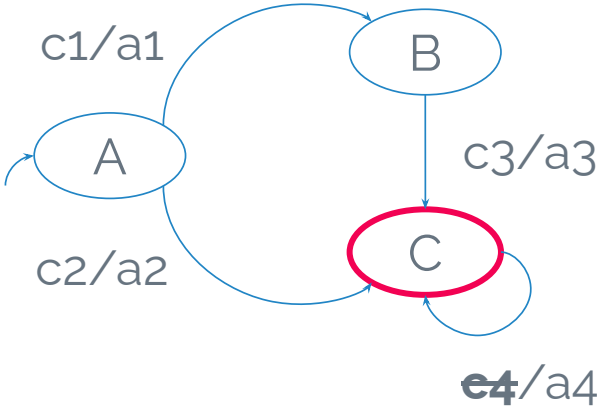
1. Comprueba en qué estado está
2. Qué posibles transiciones existen
3. Evalúa las condiciones
4. De la transición que se cumpla:
   acción + actualización del estado
5. Si no se cumple ninguna no avanza



Modelado: FSM síncrona

Update → fsm_fire(...)→ avanza la FSM síncrona

1. Comprueba en qué estado está
2. Qué posibles transiciones existen
3. Evalúa las condiciones
4. De la transición que se cumpla:
   acción + actualización del estado
5. Si no se cumple ninguna no avanza



POLITÉCNICA

Práctica A

- **Especificación**
  - Un pasillo con una bombilla controlada por un pulsador
  - Si la luz está apagada, activar el pulsador enciende la luz
  - Si la luz está encendida, activar el pulsador apaga la luz

Modelado del sistema

POLITÉCNICA

Verificación formal con LTL, promela y spin

Model Based Design (MBD)

1	2	3	4	5	6
Especificación	Modelado	Verificación	Implementación	Verificación	Análisis
¿**PARA QUÉ**, cuál es el problema?	¿**QUÉ** tiene que hacer el sistema?	¿El modelo **CUMPLE** la especificación?	¿**CÓMO** hace lo que tiene que hacer? (modelo refinado)	¿La implementación **CUMPLE** la especificación?	¿Se cumplen los **PLAZOS** en el caso peor?
LTL	FSM / xFSM	Spin (model checking)	Ejecutivo cíclico / Threads FreeRTOS	Spin (model checking)	Análisis del tiempo de respuesta

Verificación formal

- Especificación
  - Declaración del **objetivo** del diseño
  - Propiedades deseadas del sistema
- Modelado
  - **Modelo** basado en FSMs de Mealy
- Verificación
  - ¿El modelo diseñado **alcanza su objetivo** en el entorno?
  - Funcionamiento correcto → **Clave para sistemas críticos**

Verificación formal

Therac-25: Software that killed

The Therac-25, a computerized radiation therapy machine, massively overdosed patients at least six times between June 1985 and January 1987. Each overdose was several times the normal therapeutic dose and resulted in the patient's severe injury or even death.

Overdoses, although they sometimes involved operator error, **occurred primarily because of errors in the Therac-25's software and because the manufacturer did not follow proper software engineering practices.**

Verificación formal

- **LTL** (Linear Temporal Logic)
  - Describe las relaciones causales del sistema
- **Promela**
  - Lenguaje de especificación de programas concurrentes
  - Modelo promela del sistema
- **SPIN**
  - Herramienta SW de verificación formal open-source

Especificaciones → LTL

Modelo sistema → FSM

promela → spin

Verificación formal. Promela

POLITÉCNICA

- No es necesario aprender promela en profundidad
- Sólo comprender lo básico para nuestra verificación
  - Types
  - Statements
    - Asignaciones y expresiones
    - if, do, printf
  - active proctype

Verificación formal. Promela

- types → Sólo int y mtype
  - int a; ← declaración
  - int a = 1; ← declaración + inicialización
  - a = 0; ← asignación
  - mtype = { estado1, estado2 } ← nombre de estados
    - mtype asocia nombres simbólicos a valores constantes

Verificación formal. Promela

- Statements
  - El cuerpo de un proceso es una secuencia de statements
  - Un statement puede
    - Ser ejecutable ← Se puede ejecutar inmediatamente
    - Estar bloqueado ← No se puede ejecutar
      (El proceso no puede avanzar)

    Depende del estado del sistema

Verificación formal. Promela

- Statements
  - Asignaciones: Siempre ejecutables
    - b = 5
  - Expresiones: Ejecutables si se cumplen
    - 2 < 3 ← ejecutable siempre
    - 2 > 3 ← siempre se bloquea
    - b < 5 ← sólo ejecutable si se cumple la condición
    - b == 5
    - b != 5

Verificación formal. Promela

- Secuencia de statements en un proceso
  - Nota: en promela ; es un separador no un terminador
  - b=3; a= 5; c=20;
  - b < 5; b = a*2 ← Como un if en C

    Si b < 5 ejecuta b=a*2
    Si no, se queda bloqueado y el proceso no puede avanzar
    (b podría cambiar desde otro proceso)
  - if en promela funciona de otra forma

Verificación formal. Promela

- Control flow constructs
  - if (Case selection)

```
if
:: (a != b) -> b=1
:: a=1
:: c=1
:: skip
fi
```

  - p. e. explorar entorno (posibles excitaciones)

if = Estructura no determinista

Ejecuta una opción aleatoriamente (entre las que sean ejecutables)

Verificación formal. Promela

- Control flow constructs
  - do (Loop)

```
do
:: (a != b) -> b=1
:: a=1
:: c=1
:: skip
od
```

do: Estructura no determinista

Ejecuta una opción aleatoriamente <u>en bucle</u>
(igual que if pero en bucle)

Verificación formal. Promela

- Imprimir por pantalla
  - Instrucción printf ← siempre es ejecutable

```
int a = 7
int b = 0
mtype = {estado1, estado2}
int state = estado1

printf ("Valor de a es: %d \n", a)
```

Valor de a es: 7

```
printf ("a es %d, b es %d y estado es %e \n", a, b, state)
```

a es 7, b es 0 y estado es estado1

Verificación formal. Promela

- Imprimir por pantalla
  - Instrucción printf ← siempre es ejecutable

```
int a = 7
int b = 0
mtype = {estado1, estado2}
int state = estado1

printf ("Valor de a es: %d \n", a)
```

Valor de a es: 7

```
printf ("a es %d, b es %d y estado es %e \n", a, b, state)
```

a es 7, b es 0 y estado es estado1

%d valor decimal
%e constante mtype

Verificación formal. Promela

- active proctype → declaración de un proceso
  - Para las FSM del sistema y para el entorno

```
active proctype my_fsm() {
...
}
active proctype entorno() {
...
}
```

Procesos que están ejecutándose desde el principio

Estructura modelo en promela

myModel.pml

```
/*Propiedades LTL*/
ltl prop1 {formula LTL}
ltl prop2 {formula LTL} ...

/*Declarar Estados, Entradas, Salidas, Variables*/
mtype = {estado1, estado2, estado3};
int fsm1_state;
int entrada1 = 0;
int salida1 = 0;
```

Estructura modelo en promela

```
/*FSM*/

active proctype my_fsm () {
    fsm_state = estado_inicial;
    do
    :: (fsm_state == estado1) -> atomic {
          if
          :: condición -> acción/es; fsm_state = estadoX
          fi
    }
    :: (fsm_state == estado2) -> atomic {
          if
          :: transición1
          :: transición2
          fi
    }
    od
}
```

transición

Estructura modelo en promela

```
/*entorno*/

active proctype entorno (){
    do
    :: if
       :: entrada1 = 1
       :: skip -> skip
       fi ;
       printf ("variables, estados y salidas");
    od
}
```

Verificación formal. Spin

- **Verificar** → Evalúa todas las posibles trazas de ejecución

  > `spin -a myModel.pml` → Genera un verificador (pan.c)

  > `gcc -o pan pan.c` → Compila el verificador

  > `./pan -a -f -N prop1` → Ejecuta el verificador para la propiedad prop 1 y genera contraejemplo si existe (myfile.pml.trail)

  > `spin -t myModel.pml` → Simula la traza que no verifica la propiedad

Verificación formal. Spin

- Verificar → Evalúa todas las posibles trazas de ejecución

  > **spin -a myModel.pml** → Ge ador (pan.c)

  > **gcc -o pan pan.c** → Co ador

  > **./pan -a -f -N prop1** → Ejecuta el verificador para la propiedad prop 1 y genera contraejemplo si existe (myfile.pml.trail)

  > spin -t myModel.pml → Simula la traza que no verifica la propiedad

-w

(Añadir si es necesario)

Ejemplo salida Verificación formal. Spin

```
> ./pan -a -f -N enciende
...

pan: ltl formula enciende
State-vector 60 byte, depth reached 129, errors: 0

...

unreached in proctype luz_fsm
     myModel.pml:30, state 21, "-end-"
     (1 of 21 states)
unreached in proctype boton_fsm
     myModel.pml:41, state 12, "-end-"
     (1 of 12 states)
unreached in proctype entorno
     myModel.pml:51, state 10, "-end-"
     (1 of 10 states)
unreached in claim enciende
     _spin_nvr.tmp:10, state 13, "-end-"
     (1 of 13 states)
```

Los únicos estados no alcanzables son el end de los procesos

POLITÉCNICA

Práctica A

- **Especificación**
  - Un pasillo con una bombilla controlada por un pulsador
  - Si la luz está apagada, activar el pulsador enciende la luz
  - Si la luz está encendida, activar el pulsador apaga la luz

Verificación formal del sistema

POLITÉCNICA

Práctica B

- **Especificación**
  - Un pasillo con una bombilla controlada por un pulsador
  - Activar el pulsador enciende la luz
  - Si pasa un tiempo T (desde la última pulsación) la bombilla se apaga automáticamente

POLITÉCNICA

Práctica C

- **Especificación**
  - LED que parpadea periódicamente con periodo 2T
  - Si pasa un tiempo T y la bombilla está apagada la bombilla se enciende automáticamente
  - Si pasa un tiempo T y la bombilla está encendida la bombilla se apaga automáticamente

POLITÉCNICA

Práctica D

- **Especificación Sistema con dos LEDs**
  - LED controlado por un pulsador (Práctica B)
    - Activar el pulsador enciende la luz
    - Si pasa un tiempo T (desde la última pulsación) la luz se apaga automáticamente
  - LED AZUL que parpadea periódicamente con periodo 2T (Práctica C)
    - Si pasa un tiempo T y la luz azul está apagada, la luz azul se enciende automáticamente
    - Si pasa un tiempo T y la luz azul está encendida, la luz azul se apaga automáticamente

Bibliografía

- E. A. Lee and S. A. Seshia. *Introduction to Embedded Systems - A Cyber-Physical Systems Approach, Second Edition.* MIT Press, 2017
- Patrones LTL
  - https://matthewbdwyer.github.io/psp/patterns/ltl.html